DEVELOPERS HUB CORPORATION

Security Internship Phase 1 Final Project

Security Assessment and Hardening of a User Management Web Application

Student Name	***Geletaw Abebe Tsegaye***
Role	*Cybersecurity Intern*
Organization	*DevelopersHub Corporation*
Course	*Security Internship – Phase 1*
Project Duration	*March 2026*
Submission Date	*March 2026*

CONFIDENTIAL

Tools & Technologies Used

- Node.js
- Express.js
- MongoDB
- Postman (API Testing)
- Nmap (Network Scanning)
- Winston (Security Logging)
- bcrypt (Password Hashing)
- jsonwebtoken (Authentication)
- helmet (HTTP Security Header s)

Contents

Week 1 – Security Assessment Report **1**

1. Introduction **1**

1.1 Week 1 – Security Assessment 1

1.2 Application Setup 1

1.3 Vulnerability Testing 2

1.3 Injection Attack Test (SQL / NoSQL Injection) 3

1.4 Password Storage Analysis 4

1.5 Authentication Testing **6**

1.6. Summary of Findings 8

1.7. Areas for Improvement 9

1.8. Conclusion 9

2. Week 2 – Security Measures Implementation **10**

2.1 Implemented Security Measures 10

2.1.1 Input Validation & Sanitization 10

2.1.2 Password Hashing 12

2.1.3 Authentication with JWT 13

2.1.4 HTTP Security Headers (Helmet) 15

2.1.5 Role-Based Authorization 16

2.1.6 NoSQL Injection Protection 21

2.2. Summary of Findings 22

2.3. Areas for Further Improvement 23

2.4. Conclusion 23

III. Week 3 – Advanced Security and Final Reporting **24**

3. Introduction **24**

3.1. Test Environment 24

3.2 Basic Penetration Testing 24

3.3. API Security Testing 26

3.4. Advanced Logging Implementation 32

3.5. Security Best Practices Checklist 35

3.6. Conclusion **36**

Summary of Achievements in Entire Process (From Week 1 -Week 3): **37**

IV. Recommendations & Areas for Improvement **37**

V. Conclusion **37**

VI. References **38**

CONFIDENTIAL

Week 1 – Security Assessment Report

---

1. Introduction

This report compiles the work completed over **three weeks** during the Phase 1 Security Internship at the User Management System project. The internship aimed to identify and mitigate common web application vulnerabilities and strengthen the application's overall security posture.

The main objectives across the three weeks were:

- Identify and document security vulnerabilities.
- Implement essential security measures including input validation, authentication, and HTTP headers.
- Conduct penetration testing and advanced security monitoring.
- Apply best practices for secure coding and monitoring.

1.1 Week 1 – Security Assessment

Goal: Identify vulnerabilities in the User Management Web Application running locally at http://localhost:3000.

1.2 Application Setup

- Cloned application from GitHub.
- Executed locally using Node.js:

```
npm install
npm start
```

- ✓ Main tested endpoints:
  - POST /api/auth/signup
  - POST /api/auth/signin
  - GET /api/users/me

Backend Stack:

- ✓ Node.js,
- ✓ Express.js,
- ✓ MongoDB,
- ✓ JWT Authentication.

1.3 Vulnerability Testing

Several tests were performed to detect common vulnerabilities including:

- Cross-Site Scripting (XSS)
- Injection Attacks
- Password Storage Weakness
- Security Misconfiguration

Cross-Site Scripting (XSS)

- Payload tested: <script>alert('XSS')</script> in username field.
- Result: Application accepted payload → input sanitization missing.

```
//Postman
POST /api/auth/signup
```

Request Body:

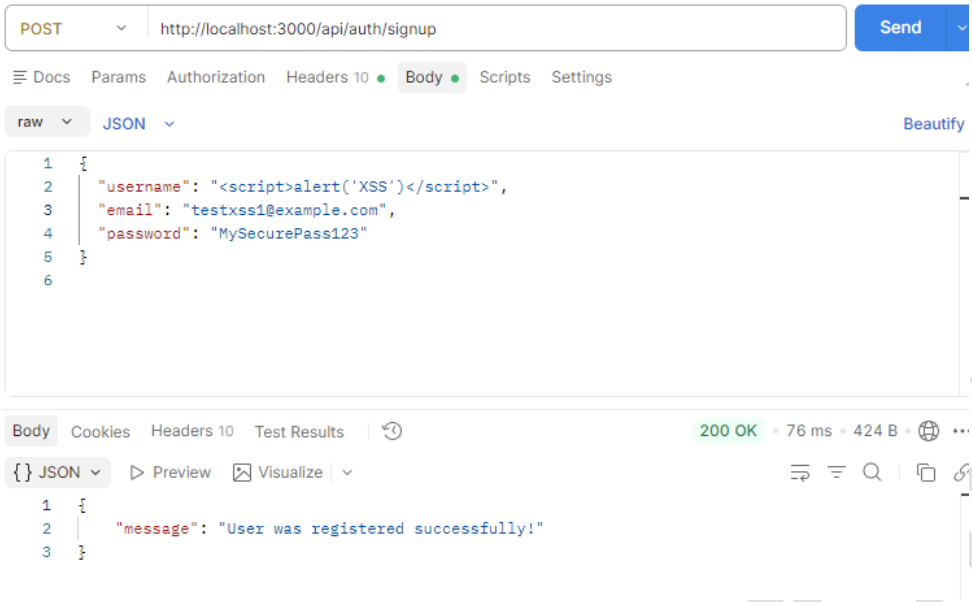
```
{
  "username": "<script>alert('XSS')</script>",
  "email": "testxss@example.com",
  "password": "MySecurePass123"
}
```

Result

The application **accepted the payload and successfully registered the user**.

This indicates that **input validation and sanitization are missing**.

CONFIDENTIAL



Security Risk

If this input is later rendered in a browser without sanitization, it could allow attackers to execute malicious JavaScript in users' browsers.

Recommendation

Input validation and sanitization should be implemented using libraries such as:

validator

Example:

```
const validator = require("validator");

if (!validator.isAlphanumeric(username)) {
  return res.status(400).send("Invalid username");
}
```

nonono

1.3 Injection Attack Test (SQL / NoSQL Injection)

Test Method

An injection payload was inserted into the username field during login testing.

Payload used:

admin' OR '1'='1

API Request:

CONFIDENTIAL

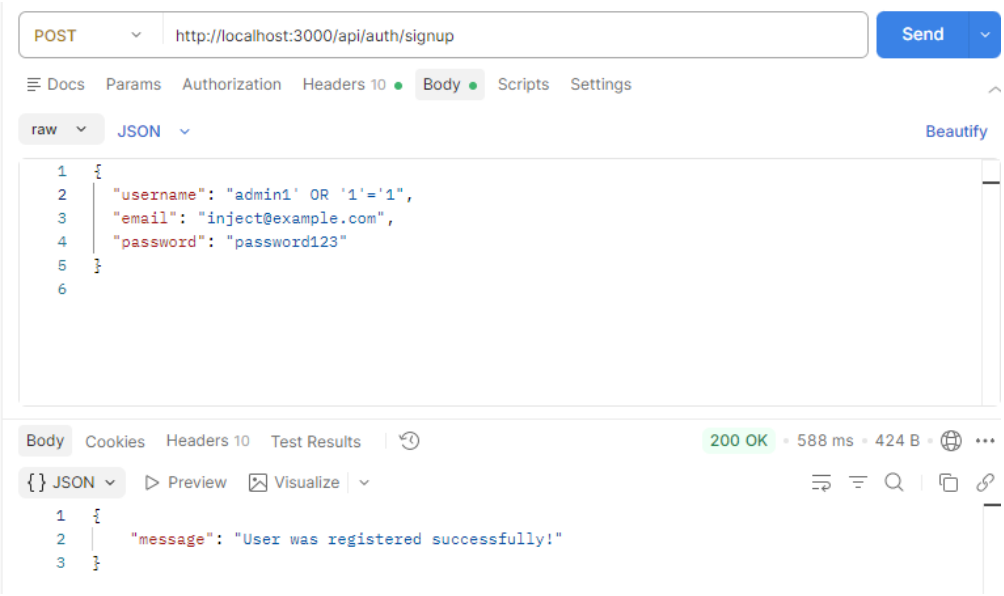
POST /api/auth/signup

Request Body:

```
{
  "username": "admin' OR '1'='1",
  "email": "inject@example.com",
  "password": "password123"
}
```

Result

The application **accepted the payload and created the user successfully**.



Security Observation

Although MongoDB is used instead of SQL, accepting such inputs without validation could lead to **NoSQL injection risks** in poorly implemented queries.

Recommendation

All user inputs should be validated and sanitized before being processed.

Example validation:

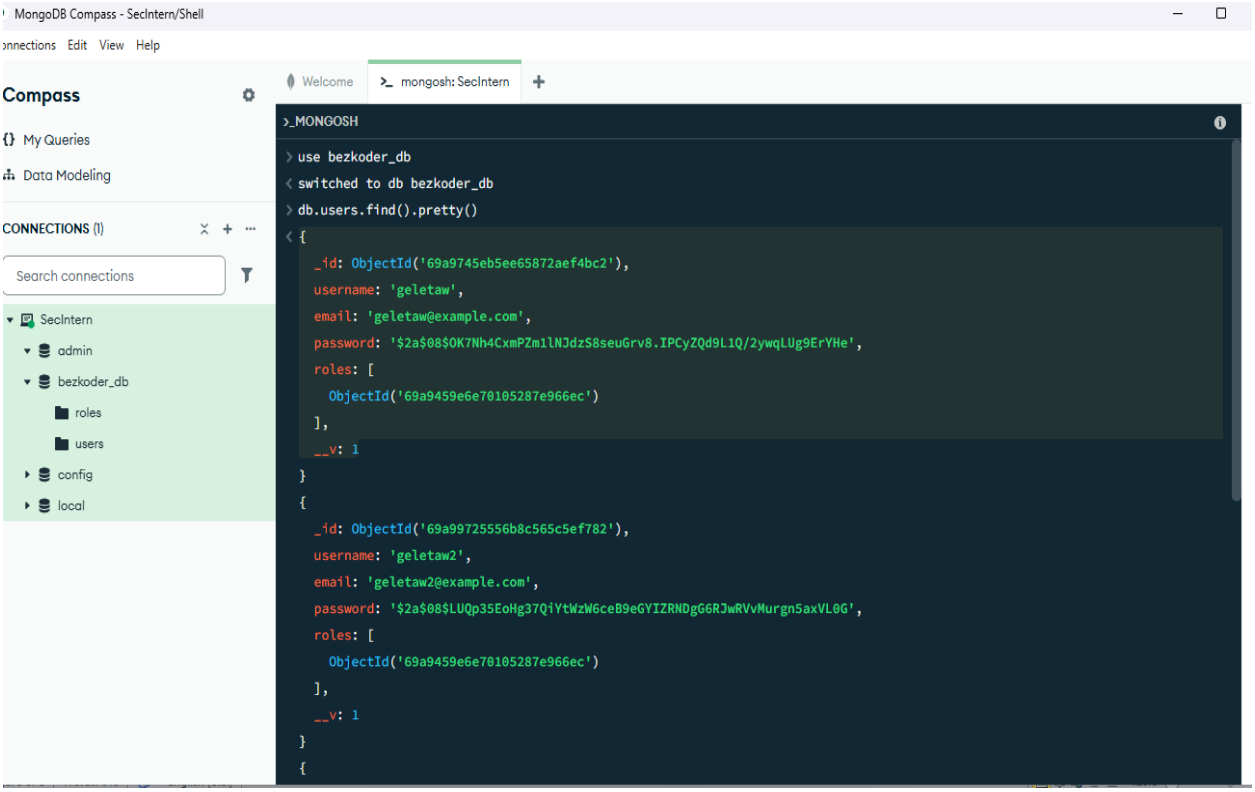
```
validator.escape(username)
```

1.4 Password Storage Analysis

The database was inspected directly in **MongoDB**.

Stored password example

$2a$08$OK7Nh4CxmPZm1lNJdzS8seuGrv8.IPCyZQd9L1Q/2ywqLUg9ErYHe



Result

- Stored passwords inspected in MongoDB.
- Passwords were securely hashed using bcrypt.
- Status**:** Secure (best practice implemented).

Security Benefit

- Protects passwords if database is compromised
- Uses salted hashing

Recommendation

Current implementation is **secure and follows best practices**.

1.5 Authentication Testing

- JWT authentication correctly implemented for user sessions.

Profile Endpoint Testing

- GET /api/users/me returned 404 → endpoint not implemented.
- **Recommendation:** Implement protected user endpoints to test role-based access control.

Login endpoint tested:

POST /api/auth/signin

Request Body:

```
{
  "username": "geletaw2",
  "password": "MySecurePass123"
}
```

Result

The application successfully returns a **JSON Web Token (JWT)**.

Example response:

```
{
  "id": "69a99725556b8c565c5ef782",
  "username": "geletaw2",
  "roles": ["ROLE_USER"],
  "accessToken": "JWT_TOKEN"
}
```

Docs Params Authorization Headers 9 Body Scripts Settings

raw JSON Beautify

```
1 {
2   "username": "test",
3   "email": "test@test.com",
4   "password": "12345678"
5 }
```

Body 200 OK 82 ms 1.2 KB

{} JSON Preview Visualize


```
1
2     "id": "69ae7e859d96163b700f07ae",
3     "username": "test",
4     "email": "test@test.com",
5     "roles": [
6         "ROLE_USER"
7     ],
8     "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY5YWU3ZTg1OWQ5NjE2M2I3MDBmMDdl
9
```


Security Observation

JWT authentication is implemented for user sessions correctly.

Profile Endpoint Testing

- GET /api/users/me returned 404 → endpoint not implemented.
- **Recommendation:** Implement protected user endpoints to test role-based access control.

Endpoint tested:

GET /api/users/me

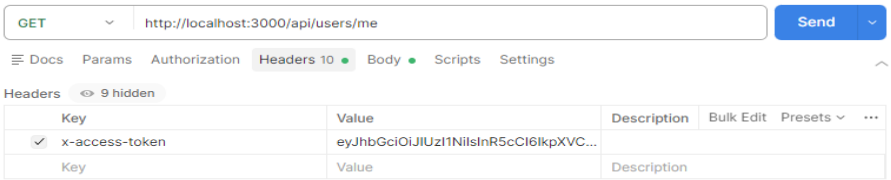
Header used:

x-access-token: JWT_TOKEN

Result

Server response:

404 Not Found
Cannot GET /api/users/me



Observation

The application does **not implement a user profile endpoint**, preventing further authorization testing.

Recommendation

Implement protected endpoints for authenticated users, such as:

GET /api/users/me

This allows testing of:

- Access control
- Role-based permissions
- Authorization security

1.6. Summary of Findings

Vulnerability	Status
Cross-Site Scripting (XSS)	Vulnerable
SQL Injection	Input validation missing
Password Storage	Secure
JWT Authentication	Implemented
Profile Endpoint	Not implemented

1.7. Areas for Improvement

The following improvements are recommended:

1. Implement input validation using **validator library**
2. Sanitize all user inputs
3. Implement **HTTP security headers using Helmet**
4. Implement profile and authorization endpoints
5. Add logging and monitoring for security events

1.8. Conclusion

The applications demonstrate basic authentication functionality and secure password storage using bcrypt. However, it lacks sufficient input validation and sanitization, making it vulnerable to potential injection and cross-site scripting attacks.

Implementing proper validation, sanitization, and additional security measures will significantly improve the application's security posture.

2. Week 2 – Security Measures Implementation

Goal: Mitigate vulnerabilities identified in Week 1.

This portion documents the security measures implemented on the User Management Web Application to address the vulnerabilities identified in Week 1. The goal was to strengthen input validation, authentication, authorization, and HTTP security headers to improve the application's security posture.

2.1 Implemented Security Measures

2.1.1 Input Validation & Sanitization

Purpose: Prevent malicious inputs such as XSS and NoSQL injection.
Implementation:

- Used the **validator** library to validate username, email, and password fields.
- Username: must be alphanumeric
- Email: must be in proper email format
- Password: minimum length 8 characters

Example:

```
const validator = require('validator');

if (!validator.isAlphanumeric(username)) {
  return res.status(400).send({ message: "Invalid username" });
}

if (!validator.isEmail(email)) {
  return res.status(400).send({ message: "Invalid email" });
}
```

Postman Test Results:

- Invalid username <script>alert('XSS')</script> → 400 Invalid username
- Invalid email → 400 Invalid email
- Valid inputs → 200 User was registered successfully

- POST http://localhost:3000/api/auth/signup Send

Docs Params Authorization Headers 10 Body Scripts Settings

raw JSON Beautify

```
1 {
2   "username": "validuser",
3   "email": "invalidemail@@example",
4   "password": "MySecurePass123"
5 }
```

Body Cookies Headers 9 Test Results 400 Bad Request 52 ms 337 B

{} JSON Preview Debug with AI

```
1 {
2   "message": "Invalid email"
3 }
```

POST http://localhost:3000/api/auth/signup Send

Docs Params Authorization Headers 10 Body Scripts Settings

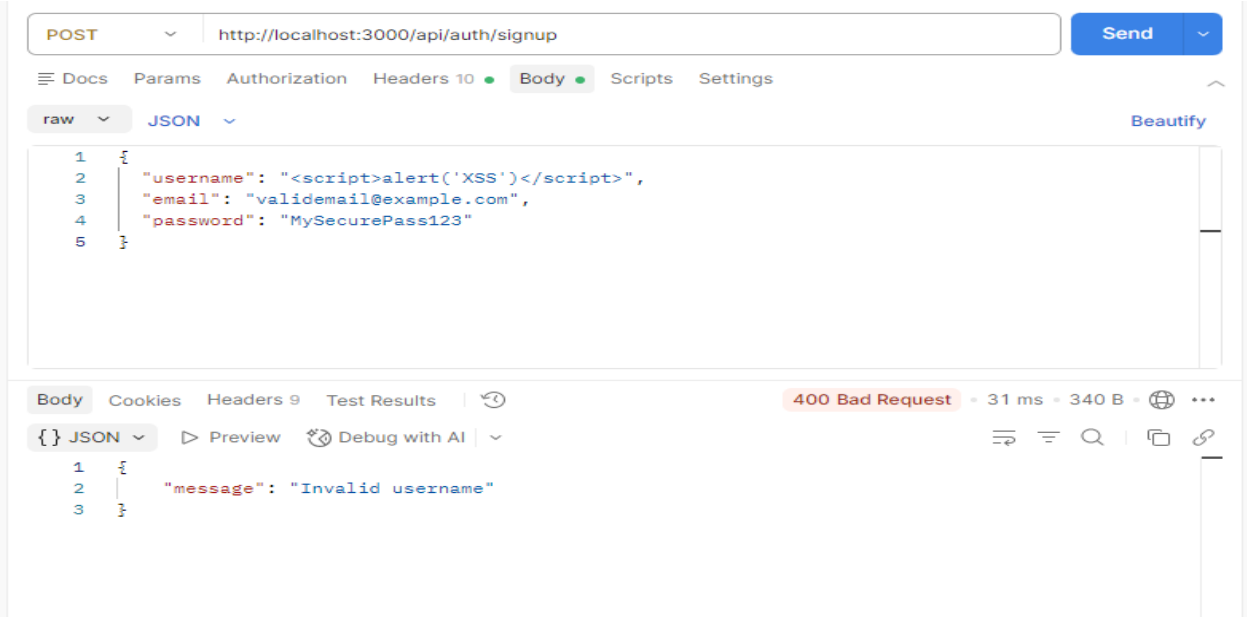
raw JSON Beautify

```
1 {
2   "username": "geletaw2",
3   "email": "geletaw2@example.com",
4   "password": "MySecurePass123"
5 }
```

Body Cookies Headers 9 Test Results 200 OK 484 ms 348 B

{} JSON Preview Visualize

```
1 {
2   "message": "User was registered successfully!"
3 }
```



2.1.2 Password Hashing

Purpose: Protect stored passwords in case of database compromise.
Implementation:

- Used **bcrypt** to hash passwords before saving to the database.
- Salt rounds: 8

Postman Verification:

- Registered user passwords in MongoDB are hashed:
  $2a$08$OK7Nh4CxmPZm1lNJdzS8seuGrv8.IPCyZQd9L1Q/...

```
>_MONGOSH
  }
  {
    _id: ObjectId('69ac32e9d960abda8c232959'),
    username: 'geletaw',
    email: 'geletaw@test.com',
    password: '$2a$08$duXl1sW2mnhWhS3o1y/oW.TJuuUxD6RwxgxZS4qutHJXJ5oZjkJ6.',
    roles: [
      ObjectId('69a9459e6e70105287e966ec')
    ],
```

Observation:

- Passwords hashed using bcrypt with 8 salt rounds.
- Verification in MongoDB confirmed hashed passwords.

2.1.3 Authentication with JWT

Purpose: Securely manage user sessions.
Implementation:

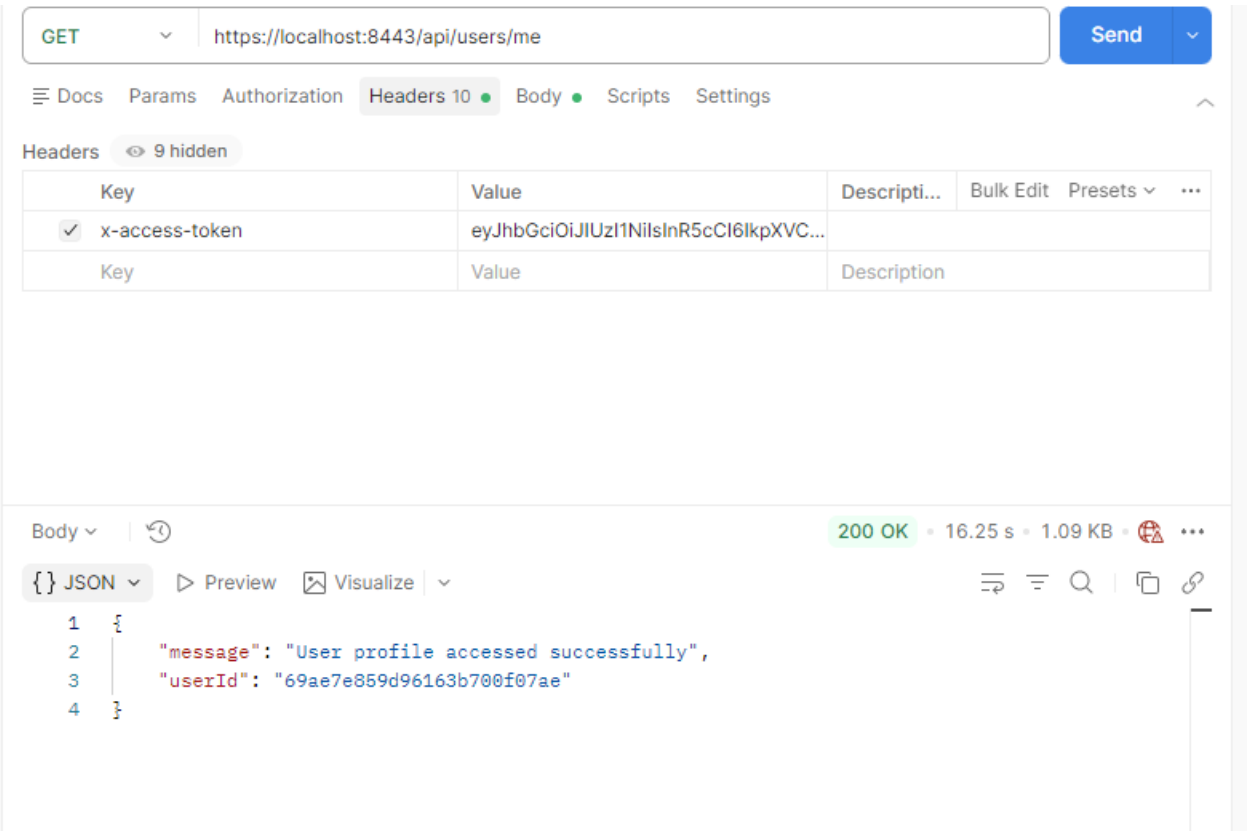
- Used **jsonwebtoken** to generate access tokens upon successful login.
- Token includes user ID and expires in 24 hours.

Example:

```
const token = jwt.sign({ id: user.id }, config.secret, { expiresIn: 86400 });
```

Postman Test Results:

- Signin returns a JWT token.
- Access to protected routes (e.g., /api/test/user) with token succeeds.
- Requests without token → 403 Forbidden



https://localhost:8443/api/users/me

Save | Share

GET https://localhost:8443/api/users/me Send

Docs | Params | Authorization | Headers 10 | Body | Scripts | Settings

Headers 9 hidden

Key	Value	Descripti...
x-access-token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC...	
Key	Value	Description

Bulk Edit | Presets

Body | 401 Unauthorized | 21.27 s | 1.04 KB

{} JSON | Preview | Pass the correct auth credentials

```
1 {
2     "message": "Unauthorized!"
3 }
```

http://localhost:3000/api/users/me

Save | Share

GET http://localhost:3000/api/users/me Send

Docs | Params | Authorization | Headers 10 | Body | Scripts | Settings

Headers 9 hidden

Key	Value	Description
x-access-token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC...	
Key	Value	Description

Bulk Edit | Presets

Body | Cookies | Headers 21 | Test Results | 200 OK | 85 ms | 1.12 KB

{} JSON | Preview | Visualize


```
1 {
2     "_id": "69a99725556b8c565c5ef782",
3     "username": "geletaw2",
4     "email": "geletaw2@example.com",
5     "roles": [
6         "69a9459e6e70105287e966ec"
7     ],
8     "__v": 1
9 }
```

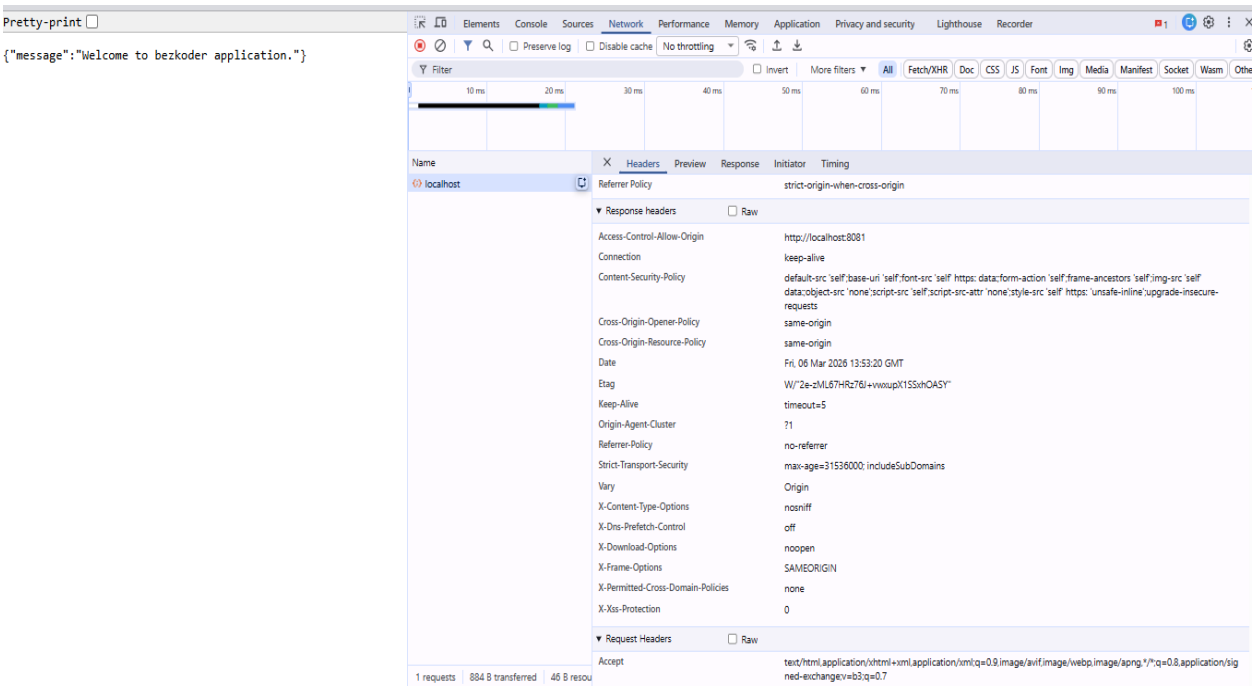
2.1.4 HTTP Security Headers (Helmet)

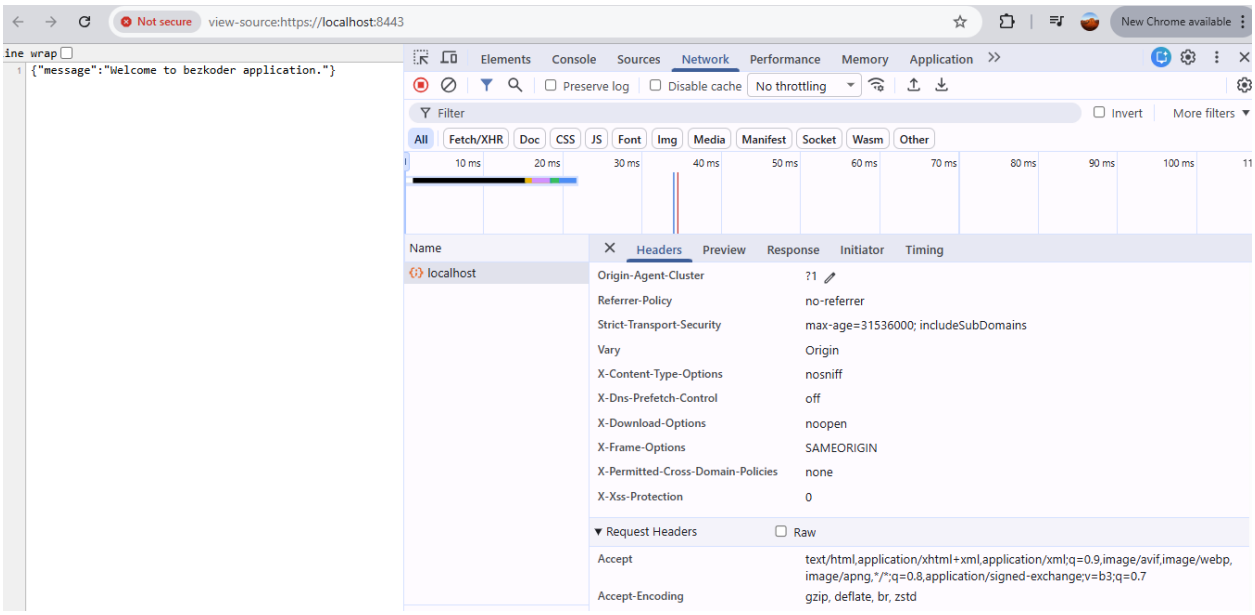
Purpose: Protect against common attacks via HTTP headers.
Implementation:

- Installed and applied **Helmet** middleware.

```
const helmet = require('helmet');
app.use(helmet());
```

- Helmet middleware applied.
- Verified headers in Developer Tools: X-Frame-Options, CSP, Strict-Transport-Security, etc.





Chrome shows **Not Secure** because you are using a **self-signed SSL certificate** for localhost.

Browsers only fully trust certificates issued by trusted authorities like:

- Let's Encrypt
- DigiCert

For **local development**, this warning is **normal and expected**.

Verified in Developer Tools (Response Headers):

- ✓ X-DNS-Prefetch-Control: off
- ✓ X-Frame-Options: SAMEORIGIN
- ✓ X-Content-Type-Options: nosniff
- ✓ Content-Security-Policy: ...
- ✓ Strict-Transport-Security: max-age=31536000; includeSubDomains

---

2.1.5 Role-Based Authorization

Purpose: Restrict access based on user roles.
Implementation:

- Protected routes using **authJwt** middleware to check token and role.
- Example routes:

  - ✓ /api/test/user,
  - ✓ /api/test/mod,
  - ✓ /api/test/admin

Postman Verification:

- User with role user cannot access admin or moderator endpoints → 403 Forbidden
- User with correct role → access granted

```
>_MONGOSH
>
>
> clear
ReferenceError: clear is not defined
>
>
>
> db.roles.find()
< {
    _id: ObjectId('69a9459e6e70105287e966ec'),
    name: 'user',
    __v: 0
  }
  {
    _id: ObjectId('69a9459e6e70105287e966ed'),
    name: 'moderator',
    __v: 0
  }
  {
    _id: ObjectId('69a9459e6e70105287e966ee'),
    name: 'admin',
    __v: 0
  }
bezkoder_db >
```

POST http://localhost:3000/api/auth/signin Send

Docs Params Authorization Headers 10 Body Scripts Settings

raw JSON Beautify

```
{
  "username": "admin1",
  "password": "AdminPassword123"
}
```

Body Cookies Headers 20 Test Results 200 OK 102 ms 1.21 KB

{} JSON Preview Visualize


```
{
    "id": "69aaf321b6fd519d02514b86",
    "username": "admin1",
    "email": "admin1@example.com",
    "roles": [
        "ROLE_ADMIN"
    ],
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY5YWFmMzIxYjZmZDUxOWQwMjUxNGI
}
```


http://localhost:3000/api/auth/signin Save Share

POST http://localhost:3000/api/auth/signin Send

Docs Params Authorization Headers 10 Body Scripts Settings

raw JSON Beautify

```
{
  "username": "mod1",
  "password": "ModPassword123"
}
```

Body Cookies Headers 20 Test Results 200 OK 63 ms 1.21 KB

{} JSON Preview Visualize


```
{
    "id": "69aaf30bb6fd519d02514b7e",
    "username": "mod1",
    "email": "mod1@example.com",
    "roles": [
        "ROLE_MODERATOR"
    ],
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY5YWFmMzBiYjZmZDUxOWQwMjUxNGI
}
```

http://localhost:3000/api/auth/signin

Save | Share

POST http://localhost:3000/api/auth/signin Send

Docs | Params | Authorization | Headers 10 | Body | Scripts | Settings

raw | JSON | Beautify

```
1 {
2   "username": "user1",
3   "password": "UserPassword123"
4 }
```

Body | Cookies | Headers 20 | Test Results | 200 OK · 71 ms · 1.2 KB

{} JSON | Preview | Visualize

```
1 {
2     "id": "69aaf2fdb6fd519d02514b7a",
3     "username": "user1",
4     "email": "user1@example.com",
5     "roles": [
6         "ROLE_USER"
7     ],
8     "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY5YWFmMmZkYjZmZDUxOWQwMjUxNGI:
9 }
```

http://localhost:3000/api/test/admin

Save | Share

GET http://localhost:3000/api/test/admin Send

Docs | Params | Authorization | Headers 10 | Body | Scripts | Settings

Headers | 9 hidden

	Key	Value	Description	Bulk Edit	Presets
✓	x-access-token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC...			
	Key	Value	Description		

Body | Cookies | Headers 21 | Test Results | 200 OK · 98 ms · 1023 B

HTML | Preview | Visualize

```
1 Admin Content.
```

http://localhost:3000/api/test/admin

Save | Share

GET http://localhost:3000/api/test/admin Send

Docs Params Authorization Headers 10 Body Scripts Settings

Headers 9 hidden

Key	Value	Description	Bulk Edit Presets
x-access-token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC...		
Key	Value	Description	

Body Cookies Headers 21 Test Results 403 Forbidden 43 ms 1.03 KB

{} JSON Preview Try with different auth credentials

```
1 {
2     "message": "Require Admin Role!"
3 }
```

o

s

is panel

http://localhost:3000/api/test/admin

Save | Share

GET http://localhost:3000/api/test/admin Send

Docs Params Authorization Headers 10 Body Scripts Settings

Headers 9 hidden

Key	Value	Description	Bulk Edit Presets
x-access-token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC...		
Key	Value	Description	

Body Cookies Headers 21 Test Results 401 Unauthorized 44 ms 1.03 KB

{} JSON Preview Pass the correct auth credentials

```
1 {
2     "message": "Unauthorized!"
3 }
```

is panel

Who Access what based on their privilege:

user1

- `/api/test/all` → ✅ Allowed
- `/api/test/user` → ✅ Allowed
- `/api/test/mod` → ❌ Forbidden
- `/api/test/admin` → ❌ Forbidden

mod1

- `/api/test/all` → ✅ Allowed
- `/api/test/user` → ✅ Allowed
- `/api/test/mod` → ✅ Allowed
- `/api/test/admin` → ❌ Forbidden

admin1

- All routes → ✅ Allowed

✓ Routes protected based on roles (user, moderator, admin).
✓ Verified access control using Postman.

2.1.6 NoSQL Injection Protection

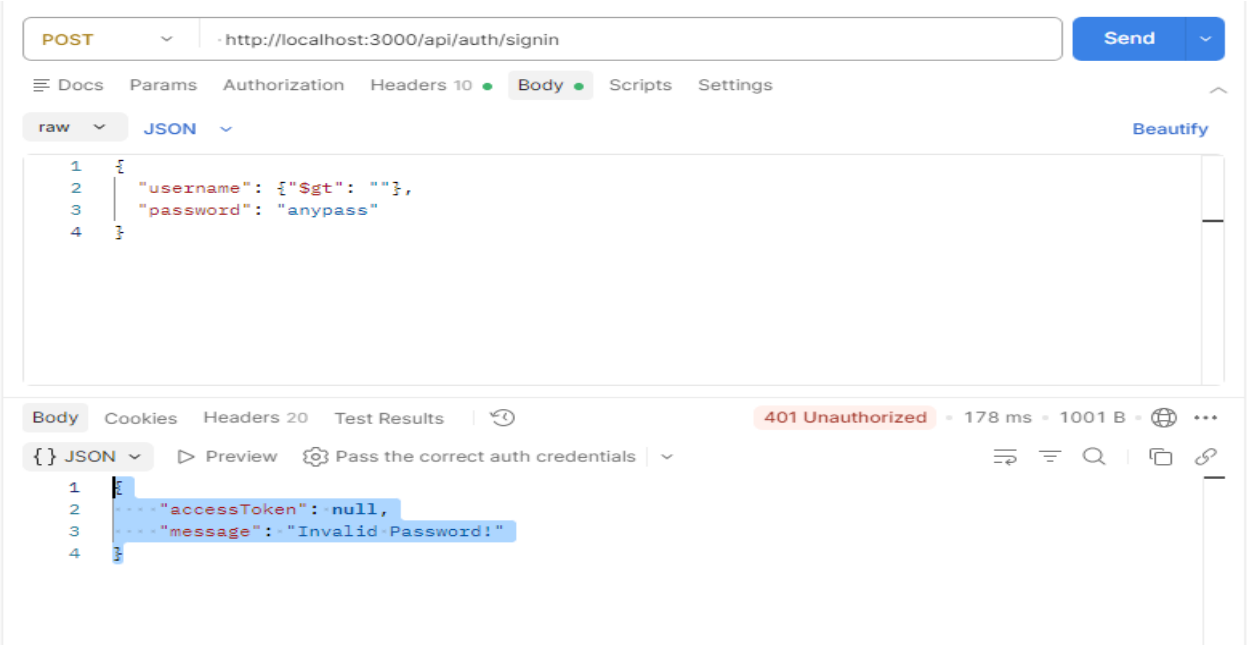
Purpose: Prevent unauthorized access via injection payloads.
Implementation:

- Input validation prevents injection payloads from being processed in MongoDB queries.

Test Example (Postman):

```
{
  "username": "admin' || '1'=='1",
  "password": "password"
}
```

- Response → User Not found.
- Response → Invalid Password! when password does not match



2.2. Summary of Findings

Vulnerability / Feature	Status
Input Validation	Implemented □
Password Hashing	Implemented □
JWT Authentication	Implemented □
HTTP Security Headers	Implemented □
Role-Based Authorization	Implemented □
NoSQL Injection	Mitigated □

2.3. Areas for Further Improvement

- Implement logging for failed login attempts.
- Apply HTTPS for secure data transmission.
- Continue monitoring security best practices and update dependencies.

2.4. Conclusion

Week 2 focused on implementing critical security measures to mitigate vulnerabilities found in Week 1. Input validation, password hashing, JWT authentication, HTTP headers, role-based access control, and NoSQL injection prevention were successfully implemented and tested. These enhancements significantly improve the security posture of the User Management System.

III. Week 3 – Advanced Security and Final Reporting

Goal: Perform advanced security testing, implement logging, and create a security checklist.

3. Introduction

During Week 3 of the cyber security internship, the focus was on implementing advanced security measures and improving the monitoring capabilities of the web application environment. This phase builds upon previous work where the application was deployed, initial vulnerabilities were assessed, and foundational security controls were introduced.

The objectives for this week included performing basic penetration testing, implementing advanced logging mechanisms for monitoring security events, and establishing a checklist of security best practices to maintain a secure application environment.

The key goals of Week 3 were:

1. Perform basic penetration testing on the local environment.
2. Implement advanced security logging for monitoring suspicious activities.
3. Create a security checklist to enforce cyber security best practices.
4. Configure HTTPS for secure data transmission.
5. Generate SSL key and certificate files for encryption.
6. Change the application server port from **3000** to **8443** for secure HTTPS communication.

3.1. Test Environment

The security testing and implementation were performed in the following environment:

- **Operating System:** Windows 11
- **Application Type:** Node.js Web Application
- **Database:** MongoDB
- **Testing Tools:** Nmap, Postman
- **Logging Library:** Winston
- **Security Libraries:** bcrypt, jsonwebtoken, helmet
- **Secure Communication Port:** 8443 (HTTPS)

3.2 Basic Penetration Testing

Tool Used

Nmap (Network Mapper)

Purpose

Network scanning was conducted to identify open ports and active services on the local system. This simulates how an attacker may enumerate services exposed on the system and identify potential entry points.

Command Used

nmap -p- 127.0.0.1

This command scans all available TCP ports on the localhost machine.

Scan Results

Port	State	Service
135	open	msrpc
137	filtered	netbios-ns
445	open	microsoft-ds
623	open	oob-ws-http
3389	open	ms-wbt-server
4439	open	unknown
4440	open	unknown
5040	open	unknown
8443	open	https-alt
15611	open	unknown
16992	open	amt-soap-http
27017	open	mongod
49664–49675	open	unknown
49748	open	unknown
53200	open	unknown
60010–60014	open	unknown
62522	open	unknown

Observations

- Several open ports correspond to core Windows services.
- Port **8443** is used by the web application for HTTPS API communication.
- Port **27017** indicates that MongoDB is running locally and may expose sensitive data if not properly secured.
- Multiple high-numbered ports appear open, which may represent dynamic or temporary services.

Security Recommendations

- Restrict open ports to only essential services.
- Apply firewall rules to limit unnecessary external access.
- Monitor unknown or high-numbered ports for suspicious activity.
- Secure database services such as MongoDB to prevent unauthorized access.

3.3. API Security Testing

Tool Used

Postman

Testing Approach

API endpoints were tested locally using Postman to simulate various attack scenarios and verify that security controls were properly implemented.

The following attack simulations were conducted:

Invalid Authentication Token

Requests were sent with invalid or missing authentication tokens to test authorization controls.

Result:
Unauthorized requests were rejected by the system.

GET https://localhost:8443/api/users/me Cancel

Docs Params Authorization Headers 11 Body Scripts Settings

Auth type Reset to Default

Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about authorization ↗

Bearer Token

Token ••••••••••••••••••••••••••••••

Body

{ } JSON Preview Pass the correct auth credentials


```
{
    "message": "Unauthorized!"
}{
    "message": "Unauthorized!"
}{
    "message": "Unauthorized!"
}
```

GET https://localhost:8443/api/users/me Send

Docs Params Authorization Headers 9 Body Scripts Settings

Headers 8 hidden

Key	Value	Descripti...
x-access-token	eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVC...	
Key	Value	Description

Bulk Edit Presets

Body Cookies Headers 21 Test Results 200 OK 819 ms 1.09 KB Save Response

{} JSON Preview Visualize


```
{
    "message": "User profile accessed successfully",
    "userId": "69ae7e859d96163b700f07ae"
}
```


GET https://localhost:8443/api/users/me Send

Docs Params Authorization Headers 10 Body Scripts Settings

Auth type Reset to Default

Bearer Token

The authorization header will be automatically generated when you send the request. Learn more about authorization ↗

Bearer Token

Token ••••••••••••••••••••••••••••••••

Body Cookies Headers 21 Test Results 200 OK 4.21 s 1.09 KB Save Response

{} JSON Preview Visualize


```
{
    "message": "User profile accessed successfully",
    "userId": "69ae7e859d96163b700f07ae"
}
```

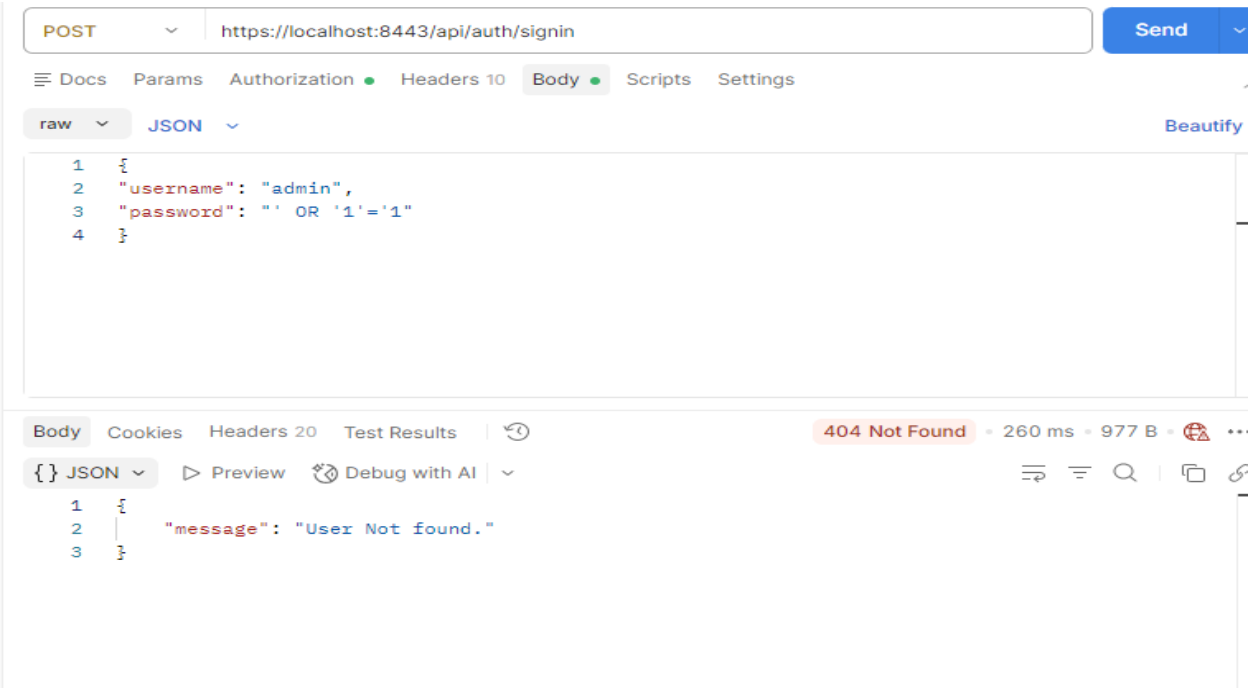
SQL Injection Testing

Payload used:

' OR '1'='1

Purpose:
Attempt to bypass authentication through SQL injection.

Result:
Input validation mechanisms successfully rejected the malicious payload.



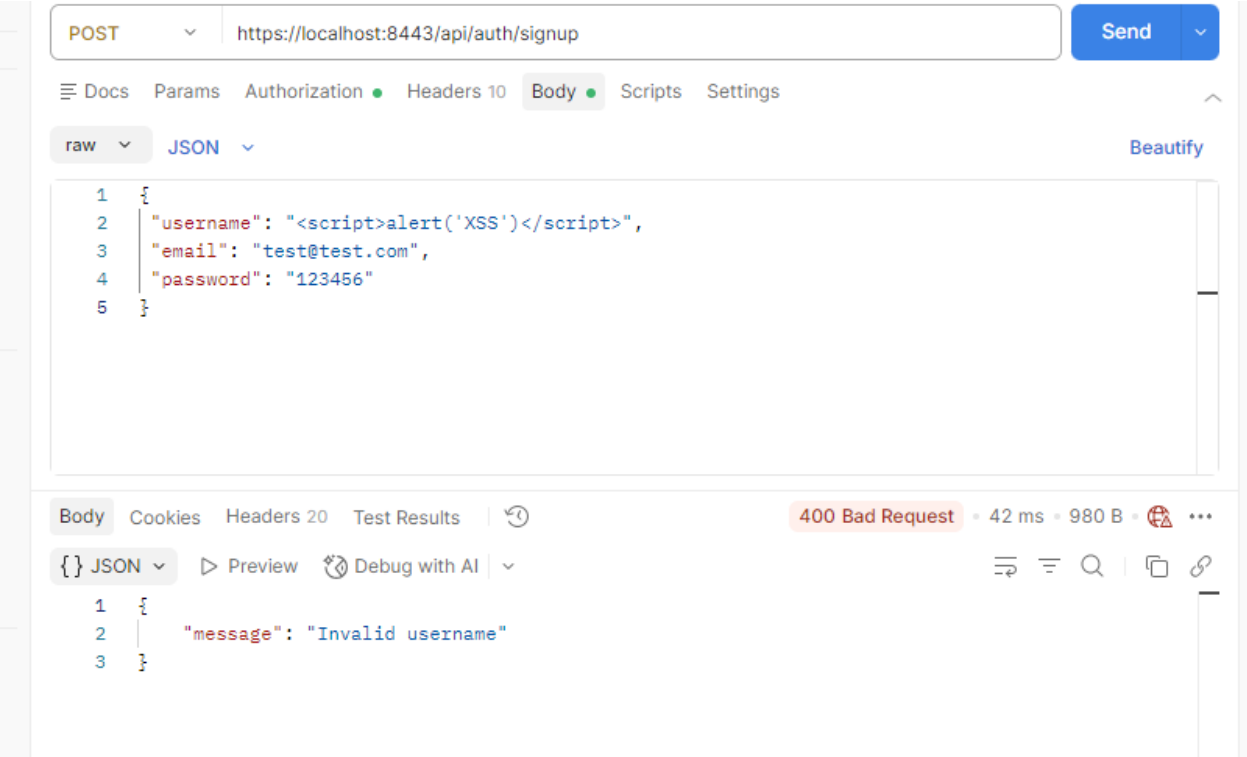
Cross-Site Scripting (XSS) Testing

Payload used:

<script>alert('xss')</script>

Purpose:
Test whether malicious scripts could be injected through user input fields.

Result:
The payload was sanitized and rejected by the application.



API Security Testing Result:

- Invalid tokens → rejected
- SQL/NoSQL injection → rejected
- XSS → sanitized and rejected
- Brute force → logged with timestamps and IPs

Brute Force Login Attempts

Multiple failed login attempts were simulated to observe system logging behavior.

POST https://localhost:8443/api/auth/signin Send

Docs Params Authorization Headers 10 Body Scripts Settings

raw JSON Beautify

```
1 {
2   "username": "gela",
3   "email": "test@test.com",
4   "password": "Geletaw123"
5 }
```

Body Cookies Headers 20 Test Results 200 OK 82 ms 1.21 KB

{} JSON Preview Visualize


```
1 {
2     "id": "69ac40b2984b89aef72a4dd3",
3     "username": "gela",
4     "email": "newuser@example.com",
5     "roles": [
6         "ROLE_USER"
7     ],
8     "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY5YWM0MGIyOTg0Yjg5YWVmNzJhNGRl
9 }
```


POST https://localhost:8443/api/auth/signin Send

Docs Params Authorization Headers 10 Body Scripts Settings

raw JSON Beautify

```
1 {
2   "username": "gela",
3   "email": "test@test.com",
4   "password": "Geletaw1223"
5 }
```

Body Cookies Headers 20 Test Results 429 Too Many Requests 34 ms 1018 B

{} JSON Preview Debug with AI

```
1 {
2     "message": "Too many failed login attempts. Try again later."
3 }
```

```
2026-03-09 10:44:51 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-09 10:44:51 [WARN] [AUTH] Failed login attempt 2 | Invalid password -> gela | ip=::1
2026-03-09 10:44:52 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-09 10:44:52 [WARN] [AUTH] Failed login attempt 3 | Invalid password -> gela | ip=::1
2026-03-09 10:44:53 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-09 10:44:53 [WARN] [AUTH] Failed login attempt 4 | Invalid password -> gela | ip=::1
2026-03-09 10:44:54 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-09 10:44:54 [WARN] [AUTH] Failed login attempt 5 | Invalid password -> gela | ip=::1
2026-03-09 10:44:54 [ERROR] [SECURITY] Possible brute force attack | user=gela | ip=::1
2026-03-09 10:44:55 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-09 10:44:55 [ERROR] [SECURITY] Brute-force lockout | user=gela | ip=::1 | attempts=5
```

Logged Result:
All login attempts were logged with timestamps, IP addresses, and usernames.

```
2026-03-09 10:44:51 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-09 10:44:51 [WARN] [AUTH] Failed login attempt 2 | Invalid password -> gela | ip=::1
2026-03-09 10:44:52 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-09 10:44:52 [WARN] [AUTH] Failed login attempt 3 | Invalid password -> gela | ip=::1
2026-03-09 10:44:53 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-09 10:44:53 [WARN] [AUTH] Failed login attempt 4 | Invalid password -> gela | ip=::1
2026-03-09 10:44:54 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-09 10:44:54 [WARN] [AUTH] Failed login attempt 5 | Invalid password -> gela | ip=::1
2026-03-09 10:44:54 [ERROR] [SECURITY] Possible brute force attack | user=gela | ip=::1
2026-03-09 10:44:55 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-09 10:44:55 [ERROR] [SECURITY] Brute-force lockout | user=gela | ip=::1 | attempts=5

Ln 371, Col 76    29,935 characters
```

3.4. Advanced Logging Implementation

To monitor suspicious activity and security events, a logging system was implemented using Winston.

Installation

npm install winston

Logging Implementation

```
const winston = require('winston');

const logFormat = winston.format.printf(({ level, message, timestamp }) => {
  return `${timestamp} [${level.toUpperCase()}]: ${message}`;
});

const logger = winston.createLogger({
  level: 'info',
  format: winston.format.combine(
    winston.format.timestamp(),
    logFormat
  ),
  transports: [
    new winston.transports.Console(),
```

```
    new winston.transports.File({ filename: 'security.log' })
  ]
});

function logSecurityEvent(eventType, username, ip, details) {
  const message = `Event: ${eventType}, User: ${username || 'unknown'}, IP: ${ip}, Details: ${details}`;
  logger.warn(message);
}
```

Example Log Output

```
2026-03-09 10:44:52 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-09 10:44:52 [WARN] [AUTH] Failed login attempt 3 | Invalid password -> gela | ip=::1
2026-03-09 10:44:53 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-09 10:44:53 [WARN] [AUTH] Failed login attempt 4 | Invalid password -> gela | ip=::1
2026-03-09 10:44:54 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-09 10:44:54 [WARN] [AUTH] Failed login attempt 5 | Invalid password -> gela | ip=::1
2026-03-09 10:44:54 [ERROR] [SECURITY] Possible brute force attack | user=gela | ip=::1
2026-03-09 10:44:55 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-09 10:44:55 [ERROR] [SECURITY] Brute-force lockout | user=gela | ip=::1 | attempts=5
```

```
2026-03-07 13:29:40 [WARN] This is a warning test
2026-03-07 13:29:40 [ERROR] This is an error test
2026-03-07 13:32:17 [INFO] User signed up successfully: john123
2026-03-07 13:40:34 [INFO] User signed up successfully: john123
2026-03-07 13:41:55 [INFO] User signed in: john123
2026-03-07 13:42:13 [WARN] Failed login attempt: Invalid password -> john123
2026-03-07 13:42:18 [WARN] Failed login attempt: Invalid password -> john123
2026-03-07 13:42:20 [WARN] Failed login attempt: Invalid password -> john123
2026-03-07 13:42:22 [WARN] Failed login attempt: Invalid password -> john123
2026-03-07 13:42:24 [WARN] Failed login attempt: Invalid password -> john123
2026-03-07 13:49:30 [WARN] Failed login attempt: Invalid password -> john123
2026-03-07 13:49:49 [WARN] Failed login attempt: Invalid password -> john123
2026-03-07 13:49:50 [WARN] Failed login attempt: Invalid password -> john123
2026-03-07 13:49:51 [WARN] Failed login attempt: Invalid password -> john123
2026-03-07 13:49:52 [WARN] Failed login attempt: Invalid password -> john123
2026-03-07 13:49:53 [WARN] Failed login attempt: Invalid password -> john123
2026-03-07 13:49:54 [WARN] Failed login attempt: Invalid password -> john123
2026-03-07 13:49:57 [WARN] Failed login attempt: Invalid password -> john123
2026-03-07 13:49:58 [WARN] Failed login attempt: Invalid password -> john123
2026-03-07 13:50:32 [INFO] Application started
2026-03-07 13:50:32 [INFO] Server is running
2026-03-07 13:50:32 [WARN] This is a warning test
2026-03-07 13:50:32 [ERROR] This is an error test
2026-03-07 13:50:52 [INFO] [AUTH] Login attempt | user=john123 | ip=::1
2026-03-07 13:50:52 [WARN] [AUTH] Failed login attempt 1 | Invalid password -> john123 | ip=::1
2026-03-07 13:50:58 [INFO] [AUTH] Login attempt | user=john123 | ip=::1
2026-03-07 13:50:58 [INFO] [AUTH] User signed in successfully | user=john123 | ip=::1
2026-03-07 13:51:32 [INFO] [AUTH] Login attempt | user=john123 | ip=::1
2026-03-07 13:51:32 [WARN] [AUTH] Failed login attempt 1 | Invalid password -> john123 | ip=::1
2026-03-07 13:51:34 [INFO] [AUTH] Login attempt | user=john123 | ip=::1
2026-03-07 13:51:34 [WARN] [AUTH] Failed login attempt 2 | Invalid password -> john123 | ip=::1
2026-03-07 13:51:35 [INFO] [AUTH] Login attempt | user=john123 | ip=::1
2026-03-07 13:51:35 [WARN] [AUTH] Failed login attempt 3 | Invalid password -> john123 | ip=::1
2026-03-07 13:51:36 [INFO] [AUTH] Login attempt | user=john123 | ip=::1
2026-03-07 13:51:36 [WARN] [AUTH] Failed login attempt 4 | Invalid password -> john123 | ip=::1
2026-03-07 13:51:36 [INFO] [AUTH] Login attempt | user=john123 | ip=::1
2026-03-07 13:51:37 [WARN] [AUTH] Failed login attempt 5 | Invalid password -> john123 | ip=::1
2026-03-07 13:51:37 [ERROR] [SECURITY] Possible brute force attack | user=john123 | ip=::1
2026-03-07 13:51:37 [INFO] [AUTH] Login attempt | user=john123 | ip=::1
2026-03-07 13:51:37 [WARN] [AUTH] Failed login attempt 6 | Invalid password -> john123 | ip=::1
2026-03-07 13:51:37 [ERROR] [SECURITY] Possible brute force attack | user=john123 | ip=::1
```

```
2026-03-07 18:58:43 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-07 18:58:43 [WARN] [AUTH] Failed login attempt 1 | Invalid password -> gela | ip=::1
2026-03-07 18:58:44 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-07 18:58:44 [WARN] [AUTH] Failed login attempt 2 | Invalid password -> gela | ip=::1
2026-03-07 18:58:44 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-07 18:58:45 [WARN] [AUTH] Failed login attempt 3 | Invalid password -> gela | ip=::1
2026-03-07 18:58:46 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-07 18:58:46 [WARN] [AUTH] Failed login attempt 4 | Invalid password -> gela | ip=::1
2026-03-07 18:58:47 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-07 18:58:47 [WARN] [AUTH] Failed login attempt 5 | Invalid password -> gela | ip=::1
2026-03-07 18:58:47 [ERROR] [SECURITY] Possible brute force attack | user=gela | ip=::1
2026-03-07 18:58:48 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-07 18:58:48 [ERROR] [SECURITY] Brute-force lockout | user=gela | ip=::1 | attempts=5
2026-03-07 18:58:48 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-07 18:58:48 [ERROR] [SECURITY] Brute-force lockout | user=gela | ip=::1 | attempts=5
2026-03-07 18:58:49 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-07 18:58:49 [ERROR] [SECURITY] Brute-force lockout | user=gela | ip=::1 | attempts=5
2026-03-07 18:58:50 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-07 18:58:50 [ERROR] [SECURITY] Brute-force lockout | user=gela | ip=::1 | attempts=5
2026-03-07 18:58:51 [INFO] [AUTH] Login attempt | user=gela | ip=::1
2026-03-07 18:58:51 [ERROR] [SECURITY] Brute-force lockout | user=gela | ip=::1 | attempts=5
2026-03-07 19:13:18 [INFO] [AUTH] Login attempt | user=admin | ip=::1
2026-03-07 19:13:18 [WARN] [AUTH] Failed login attempt 1 | User not found -> admin | ip=::1
2026-03-07 19:13:20 [INFO] [AUTH] Login attempt | user=admin | ip=::1
2026-03-07 19:13:20 [WARN] [AUTH] Failed login attempt 2 | User not found -> admin | ip=::1
2026-03-07 19:13:21 [INFO] [AUTH] Login attempt | user=admin | ip=::1
2026-03-07 19:13:21 [WARN] [AUTH] Failed login attempt 3 | User not found -> admin | ip=::1
2026-03-07 19:13:22 [INFO] [AUTH] Login attempt | user=admin | ip=::1
2026-03-07 19:13:22 [WARN] [AUTH] Failed login attempt 4 | User not found -> admin | ip=::1
2026-03-07 19:13:23 [INFO] [AUTH] Login attempt | user=admin | ip=::1
2026-03-07 19:13:23 [WARN] [AUTH] Failed login attempt 5 | User not found -> admin | ip=::1
2026-03-07 19:13:23 [ERROR] [SECURITY] Possible brute force attack | user=admin | ip=::1
2026-03-07 19:13:24 [INFO] [AUTH] Login attempt | user=admin | ip=::1
2026-03-07 19:13:24 [ERROR] [SECURITY] Brute-force lockout | user=admin | ip=::1 | attempts=5
```

Observations

- Security events are recorded with timestamps, IP addresses, usernames, and event details.
- Logging captures multiple types of attacks including brute-force attempts, SQL injection, and XSS.
- Security logs provide valuable information for incident analysis and forensic investigations.

3.5. Security Best Practices Checklist

Security Measure	Status / Notes
Validate all inputs	Implemented and tested
Use HTTPS for all data transmission	Verified on port 8443
Hash and salt passwords	Implemented in authentication module
Restrict unnecessary open ports	Pending firewall configuration
Monitor logs for suspicious activity	Implemented using Winston
Detect brute-force attempts	Logged with timestamps and IP
Detect SQL injection attempts	Payload detected and logged
Detect XSS attacks	Input sanitized and logged
Test API authentication and error responses	Successfully validated

3.6. Conclusion

Week 3 successfully introduced advanced security controls to improve the protection and monitoring of the web application environment.

Network scanning using Nmap helped identify open ports and potential attack surfaces, while API testing with Postman allowed simulation of real-world attack scenarios including SQL injection, XSS attacks, and unauthorized access attempts.

The implementation of Winston logging provides a strong foundation for security monitoring by recording critical events such as login attempts, suspicious payloads, and unauthorized access activities.

Additionally, enabling HTTPS communication on port **8443** ensures secure data transmission between clients and the server. The security checklist developed during this phase serves as a practical guide to maintain proper security hygiene and prevent common web application vulnerabilities.

Overall, the combination of penetration testing, secure configuration, and event logging significantly strengthens the security posture of the application environment.

Summary of Achievements in Entire Process (From Week 1 -Week 3):

- **Week 1**: Identified vulnerabilities such as XSS, injection risks, and missing profile endpoints.
- **Week 2**: Implemented input validation, password hashing, JWT authentication, HTTP headers, role-based access, and injection protection.
- **Week 3**: Performed penetration testing, implemented advanced logging, configured HTTPS, and created a comprehensive security checklist.

IV. Recommendations & Areas for Improvement

- Apply firewall rules to limit open ports.
- Enforce HTTPS for all production environments.
- Continue monitoring logs and failed login attempts.
- Expand penetration testing to include automated scanners and third-party tools.
- Keep dependencies updated and maintain security best practices documentation.

V. Conclusion

Over the three-week internship, significant progress was made in identifying, mitigating, and preventing common web application vulnerabilities. Implementing input validation, secure password storage, JWT authentication, HTTP security headers, role-based access control, and logging mechanisms has strengthened the application's security posture.

The combination of penetration testing, secure coding practices and logging ensures the system is more resilient to attacks and provides a foundation for future security improvements. This internship has provided practical hands-on experience in cyber security for web applications and laid a solid groundwork for ongoing security management.

VI. References

The following resources were used to understand web application security concepts, testing techniques, and implementation of security controls.

1. OWASP Foundation.
   OWASP Top 10 Web Application Security Risks.
   https://owasp.org/www-project-top-ten/
2. Node.js Official Documentation.
   https://nodejs.org/en/docs/
3. Express.js Documentation.
   https://expressjs.com/
4. MongoDB Documentation.
   https://www.mongodb.com/docs/
5. Postman API Platform Documentation.
   https://learning.postman.com/
6. Nmap Official Guide.
   https://nmap.org/book/
7. Winston Logging Documentation.
   https://github.com/winstonjs/winston
8. OWASP Foundation Secure Coding Practices.
   https://owasp.org/www-project-secure-coding-practices/